

Bulk Collection with Log Tables

DML Error Logging (LOG ERRORS) & custom exception logging · Oracle 19c

When loading large volumes, you don't want one bad row to roll back the entire batch. Two complementary techniques capture failures into a **log table** while good rows still commit: the built-in **LOG ERRORS** clause, and a **custom error-log** pattern using **SAVE EXCEPTIONS**.

1. Create the Error Log Table

Oracle can generate a matching error-log table automatically with **DBMS_ERRLOG**:

```
BEGIN
  DBMS_ERRLOG.create_error_log(
    dml_table_name => 'EMP_TGT',           -- base table
    err_log_table_name => 'ERR$_EMP_TGT'); -- optional custom name
END;
/
```

This creates **ERR\$_EMP_TGT** with standard columns plus a VARCHAR2 copy of every base column:

Column	Meaning
ORA_ERR_NUMBER\$	Oracle error number (e.g. 1 for unique violation)
ORA_ERR_MESG\$	Error message text
ORA_ERR_ROWID\$	ROWID of the failing row (for updates/deletes)
ORA_ERR_OPTYP\$	Operation: I / U / D / M
ORA_ERR_TAG\$	Your custom tag string per statement

2. LOG ERRORS Clause (works with FORALL)

```
DECLARE
  TYPE t_emp IS TABLE OF emp_src%ROWTYPE;
  l_emp t_emp;
BEGIN
  SELECT * BULK COLLECT INTO l_emp FROM emp_src;

  FORALL i IN 1 .. l_emp.COUNT
    INSERT INTO emp_tgt VALUES l_emp(i)
    LOG ERRORS INTO err$_emp_tgt ('LOAD_' || TO_CHAR(SYSDATE, 'YYYYMMDD'))
    REJECT LIMIT UNLIMITED; -- keep going past all errors

  COMMIT;
END;
/
```

Good rows are inserted & committed; bad rows land in **ERR\$_EMP_TGT** with the reason. Review them:

```
SELECT ora_err_number$, ora_err_mesg$, ora_err_tag$, id, sal
FROM   err$_emp_tgt
ORDER BY ora_err_tag$;
```

REJECT LIMIT controls how many errors are tolerated before the statement itself fails. Use **UNLIMITED** for data loads; use a small number (e.g. **REJECT LIMIT 100**) to abort if too many rows are bad.

3. Custom Log Table Pattern (SAVE EXCEPTIONS)

When you want your own structured log (with timestamps, job id, etc.), combine **FORALL ... SAVE EXCEPTIONS** with a custom logging table.

```
CREATE TABLE load_error_log (  
  log_id      NUMBER GENERATED BY DEFAULT AS IDENTITY,  
  job_name    VARCHAR2(60),  
  err_index   NUMBER,  
  err_code    NUMBER,  
  err_msg     VARCHAR2(400),  
  key_value   VARCHAR2(100),  
  logged_at   TIMESTAMP DEFAULT SYSTIMESTAMP);  
  
DECLARE  
  TYPE t_emp IS TABLE OF emp_src%ROWTYPE;  
  l_emp      t_emp;  
  bulk_errors EXCEPTION;  
  PRAGMA EXCEPTION_INIT(bulk_errors, -24381); -- ORA-24381: FORALL had errors  
BEGIN  
  SELECT * BULK COLLECT INTO l_emp FROM emp_src;  
  
  FORALL i IN 1 .. l_emp.COUNT SAVE EXCEPTIONS  
    INSERT INTO emp_tgt VALUES l_emp(i);  
  
  COMMIT;  
EXCEPTION  
  WHEN bulk_errors THEN  
    FOR j IN 1 .. SQL%BULK_EXCEPTIONS.COUNT LOOP  
      DECLARE  
        idx PLS_INTEGER := SQL%BULK_EXCEPTIONS(j).error_index;  
      BEGIN  
        INSERT INTO load_error_log (job_name, err_index, err_code, err_msg, key_value)  
          VALUES ('EMP_LOAD', idx,  
                SQL%BULK_EXCEPTIONS(j).error_code,  
                SQLERRM(-SQL%BULK_EXCEPTIONS(j).error_code),  
                l_emp(idx).id);  
      END;  
    END LOOP;  
    COMMIT; -- commit good rows + the log  
END;  
/
```

4. Which approach to use?

Aspect	LOG ERRORS clause	SAVE EXCEPTIONS + custom log
Setup	One call to DBMS_ERRLOG	Build your own table & handler
Captured data	Full failing row (as text)	Whatever you choose to log
Best for	Data loads / migrations	Custom audit, job tracking
Control	REJECT LIMIT threshold	Full procedural control

Tips

- **LOG ERRORS** does **not** capture errors that occur before row processing (e.g. a bad datatype conversion in the SELECT).
- Error-log columns are **VARCHAR2(4000)** copies — very long values may be truncated.
- With **SAVE EXCEPTIONS**, **SQL%BULK_EXCEPTIONS(j).error_index** maps back to the collection subscript that failed.
- Always **COMMIT** after logging so the successful rows and the log entries persist together.